



Master 1 GIL - XML
Projet : Service REST & Compléments

RAPPORT MINI PROJET

Version : 1.0

Date : 15/04/2026

Rédigé par : Umm-Habibah Ouattara et Florian Pépin

Table des matières

1	Introduction	2
2	Identification du Projet	2
3	Architecture du Projet	3
3.1	Organisation Logicielle	3
3.1.1	Détail des composants principaux	3
3.2	Stack Technique	4
3.3	Points d'accès (Endpoints)	4
4	Répartition du Travail	5
5	Description de l'usine logicielle	6
5.1	Surveillance et Déclenchement	6
5.2	Chaîne de Build et Automatisation	6
5.3	Sécurité et Isolation	6
5.4	Maintenance et Redémarrage	7
6	Éléments Optionnels	8
7	Identifiants et Accès aux Services	9
7.1	Identifiants des utilisateurs (VM)	9
7.2	Accès Jenkins	9
7.3	Service REST SEPA26	9

1 Introduction

Ce projet porte sur la réalisation du service REST SEPA26. L'objectif est de construire une architecture basé sur la norme ISO 20022 fourni en cours pour gérer des transactions. Le déploiement est automatisé sur une VM via Jenkins.

2 Identification du Projet

- **Étudiants** : Umm-Habibah Ouattara et Florian Pépin
- **Nom de la VM** : `vm-pepinflo-xml.univ-rouen.fr`
- **Adresse IP** : `10.130.162.188`
- **URL de l'application** : `http://10.130.162.188:8100/`

3 Architecture du Projet

L'application est structurée selon un modèle classique en couches (Layered Architecture), garantissant une séparation claire des responsabilités.

3.1 Organisation Logicielle

L'application suit une architecture en couches, dont voici les responsabilités principales :

- **Couche de Présentation (Web/API) :** Gère les requêtes HTTP via les contrôleurs et assure le rendu des vues.
- **Couche de Service :** Centralise la logique métier et les règles de gestion.
- **Couche d'Accès aux Données :** Assure la persistance via Spring Data JPA.
- **DTO & Utilitaires :** Facilitent le transfert de données et les transformations XML/XSLT.

3.1.1 Détail des composants principaux

Classe / Composant	Rôle et Responsabilité
Sepa26Application	Point d'entrée principal qui démarre l'application Spring Boot.
SepaService	Centralise la logique métier et coordonne les opérations sur les transactions.
Document	Entité représentant une transaction SEPA et sa structure en base de données.
DocumentRepository	Interface assurant le lien entre l'application et la base de données MariaDB.
GetController	Gère la récupération et l'affichage des transactions (XML et HTML).
PostController	Traite l'insertion de nouvelles transactions et les demandes de suppression.
SearchController	Propose des fonctionnalités de recherche multicritères sur les transactions.
IndexController	Gère la navigation vers la page d'accueil et la documentation d'aide.
TransfertController	Fournit l'interface pour l'importation de fichiers XML par l'utilisateur.
XsltTransformer	Transforme les données XML brutes en pages HTML lisibles via des feuilles XSLT.
DocumentList	DTO utilisé pour formater une collection de documents dans les réponses XML.
SepaResponse	Structure standardisée pour les messages de retour (succès/erreur) de l'API.

Classe / Composant	Rôle et Responsabilité
DataInitializer	Initialise la persistance avec des données de test lors du démarrage du serveur.

3.2 Stack Technique

Le service repose sur une stack Java 21 moderne et robuste :

- **Framework** : Spring Boot (Spring Web, Spring Data JPA).
- **Gestion des dépendances** : Maven.
- **Stockage** : MariaDB (Production) et H2 (Tests).
- **Traitement XML** : JAXB, Jackson-XML et Saxon-HE (XSLT/XPath).
- **Interface** : Thymeleaf pour le rendu des vues HTML dynamiques.

3.3 Points d'accès (Endpoints)

Le tableau suivant récapitule l'ensemble des points d'entrée du service (API REST et Interface Web) accessibles sous le préfixe `/sepa26` :

Méthode	Endpoint	Description / Format de sortie
GET	/	Page d'accueil du service
GET	/help	Page d'aide et documentation de l'API
GET	/transfert	Interface d'importation de fichiers XML
POST	/insert	Création d'un virement (Flux XML ISO 20022)
DELETE	/delete/{id}	Suppression d'une transaction (Retour XML)
GET	/resume/xml	Liste de toutes les transactions (XML)
GET	/resume/html	Liste de toutes les transactions (HTML)
GET	/xml/{id}	Détail complet d'une transaction (XML)
GET	/html/{id}	Détail complet d'une transaction (HTML)
GET	/search	Recherche avancée multicritères (XML)

TABLE 2 – Récapitulatif des services et pages web

4 Répartition du Travail

Le développement du projet a été équitablement réparti entre les deux membres du binôme. Le tableau ci-dessous détaille la distribution des modules selon les sections du cahier des charges :

Développeur	Modules et Tâches réalisées
Florian Pépin	— Développement des contrôleurs principaux (Accueil I.1, Aide I.2, Résumés I.3, Détails I.4 et Insertion I.5). — Bonus : Gestion optimisée des messages d’erreurs (II.1). — Infrastructure : Mise en place de la persistance (Base de données II.3).
Umm-Habibah Ouattara	— Mise en œuvre des contraintes du service et validation XSD (I.6). — Développement de l’API de recherche (II.2). — Qualité : Tests d’intégration et création de la collection Postman (II.3). — Outil : Développement de l’application de transfert de flux (III.1).

TABLE 3 – Répartition détaillée des tâches

5 Description de l’usine logicielle

Afin d’assurer une livraison fluide et automatisée du service SEPA26, nous avons mis en place une usine logicielle complète intégrant l’Intégration Continue (CI) et le Déploiement Continu (CD).

5.1 Surveillance et Déclenchement

L’usine logicielle s’appuie sur une scrutation active (Polling SCM) du dépôt de gestion de version. Jenkins vérifie chaque minute (* * * * *) si de nouvelles modifications ont été apportées au code source. Si un changement est détecté, le cycle de build est automatiquement lancé.

5.2 Chaîne de Build et Automatisation

Le processus est orchestré par un `Jenkinsfile` (Pipeline) structuré en plusieurs étapes clés :

- **Préparation** : Initialisation de l’environnement, création d’un réseau Docker dédié (`sepa-network`) pour isoler les communications et nettoyage des anciens conteneurs.
- **Gestion des Données (MariaDB)** : Lancement automatique d’un conteneur MariaDB. La persistance est assurée par un volume Docker et les accès sont sécurisés via l’utilisation des *Credentials* Jenkins pour masquer les mots de passe.
- **Compilation et Packaging (Maven)** : Exécution de la commande `mvn package` qui assure la compilation, le passage des tests unitaires et d’intégration, et la génération de l’artefact exécutable.
- **Conteneurisation (Docker)** : Construction de l’image Docker de l’application intégrant l’artefact généré.
- **Déploiement** : Lancement du conteneur applicatif sur le port 8100, lié dynamiquement à la base de données MariaDB via les variables d’environnement.

5.3 Sécurité et Isolation

L’architecture de l’usine logicielle privilégie la sécurité :

- **Secrets** : Aucun mot de passe n’est stocké en clair dans les scripts ; Jenkins injecte dynamiquement les secrets au moment de l’exécution.
- **Réseau** : L’utilisation d’un réseau Docker *bridge* assure que seul le serveur applicatif est exposé à l’extérieur, la base de données n’étant accessible que par l’application au sein du réseau privé Docker.

5.4 Maintenance et Redémarrage

Grâce à l'automatisation via Jenkins et Docker, le redémarrage ou la mise à jour du service est simplifié :

- **Redémarrage via Jenkins :** Une simple action "*Lancer un build*" sur l'interface Jenkins suffit à déclencher le pipeline complet. Le script se charge de stopper les instances existantes, de recompiler le code et de relancer proprement les conteneurs MariaDB et SEPA26.
- **Redémarrage manuel :** En cas de besoin, il est également possible de redémarrer le service manuellement sur la VM via les commandes Docker (`docker restart servicesepa26`).

6 Éléments Optionnels

Nous avons ajouté quelques fonctionnalités pour rendre le service plus simple à utiliser depuis un navigateur :

- **Formulaire d’insertion** : Il est possible d’envoyer un fichier XML directement depuis la page `/transfert` sans avoir à passer par une commande `curl`.
- **Bouton supprimer** : Sur la page des résumés (`/resume/html`), nous avons ajouté un bouton pour supprimer une transaction en un clic. C’est plus pratique que de forger une requête `DELETE` à la main.
- **Collection Postman** : Nous avons fourni le fichier `sepa26_pepin_ouattara.json` qui contient déjà toutes les requêtes prêtes à l’emploi pour tester le serveur (VM).

7 Identifiants et Accès aux Services

Cette section regroupe les identifiants nécessaires pour accéder aux différents outils de l'usine logicielle et tester le service.

7.1 Identifiants des utilisateurs (VM)

Usage	Utilisateur	Mot de passe
Admin	spring	abcd1230
Projet REST	user-git	user-git
Tests sur la VM	test	test

TABLE 4 – Identifiants de la VM

7.2 Accès Jenkins

- URL : `http://10.130.162.188:8080`
- Login : `jobadmin`
- Mot de passe : `JobAdmin26`

7.3 Service REST SEPA26

- Interface Web : `http://10.130.162.188:8100/`